

VUE • FLASK • POSTGRESQL • REDIS • AI

# 电商 AI 商品文案生成与智能导购助手

## 系统架构与维护设计

单页应用

分层 API

数据与缓存

AI Provider

项目组（成员姓名待补充） • 2026-07-15

# 系统组成

## Vue 3

浏览器单页应用

## Flask 3

REST API 服务

## PostgreSQL

业务数据持久化

## Redis 7

重复生成缓存

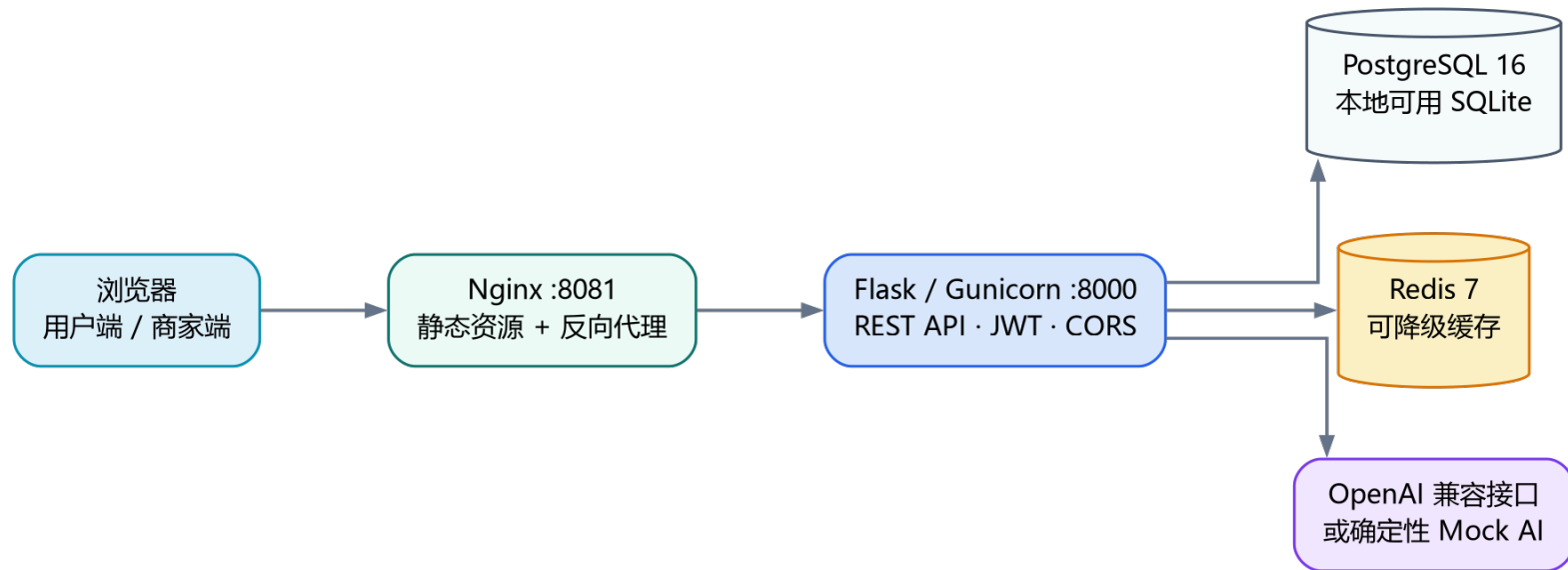
## 开发运行

Vue 由 Vite 提供开发服务器；Flask 运行在 8000；未设置数据库地址时使用 `instance/ecommerce_ai.db`。

## 容器运行

Nginx 暴露 8081；Flask 由 Gunicorn 托管；PostgreSQL 与 Redis 由 Compose 编排。

# 系统边界与访问路径



静态站点

业务接口

健康接口

/ 由 Nginx 返回 Vue 构建产物。

/api/\* 反向代理到 Flask :8000。

/health 直达 Flask 健康检查。

# 技术栈分层

| 层次  | 主要技术                               | 职责                |
|-----|------------------------------------|-------------------|
| 前端  | Vue 3 · TypeScript · Vite 8        | 页面组织、状态恢复、异步组件与交互 |
| UI  | Ant Design Vue · Tailwind · Motion | 标准控件、布局规则、动画反馈与主题 |
| API | Flask 3 · Pydantic 2 · Gunicorn    | 路由、参数校验、业务编排与生产托管 |
| 安全  | JWT · Werkzeug · Flask-CORS        | 身份、角色、密码哈希与来源控制   |
| 数据  | SQLAlchemy · Alembic · PostgreSQL  | 模型映射、会话管理与版本迁移    |
| 智能  | Redis · OpenAI-compatible · Mock   | 缓存、RAG 上下文与内容生成   |

# 目录职责

backend/api/

**api**: 按认证、用户、商家、客服、采集拆分蓝图。

backend/models/

**models**: SQLAlchemy 领域模型与关系。

backend/services/

**services**: AI、RAG、认证、上传等业务服务。

backend/crawler/

**crawler**: 京东、苏宁采集和任务管理。

frontend/src/

**frontend**: Vue 页面、组件、主题与 API 客户端。

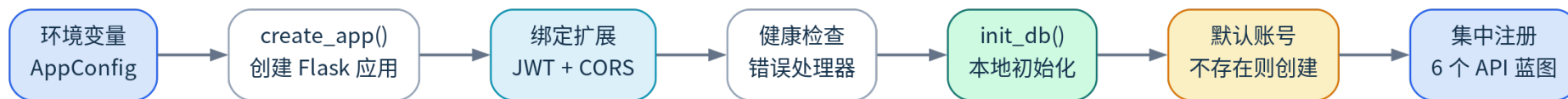
migrations/

**migrations**: Alembic 版本迁移。

**scripts**: 启动与数据维护入口。

scripts/

# 后端启动流程



`backend.app:create_app()` 统一完成配置装载、扩展绑定、错误处理、数据库初始化、账号初始化和蓝图注册。



# 应用工厂的职责边界

## 配置

从环境变量构造不可变的 AppConfig。

## 框架扩展

JWTManager 与 CORS 在创建应用时绑定。

## 基础设施

健康检查、400/404/500 错误处理器。

## 本地初始化

init\_db() 保障 SQLite 首次运行。

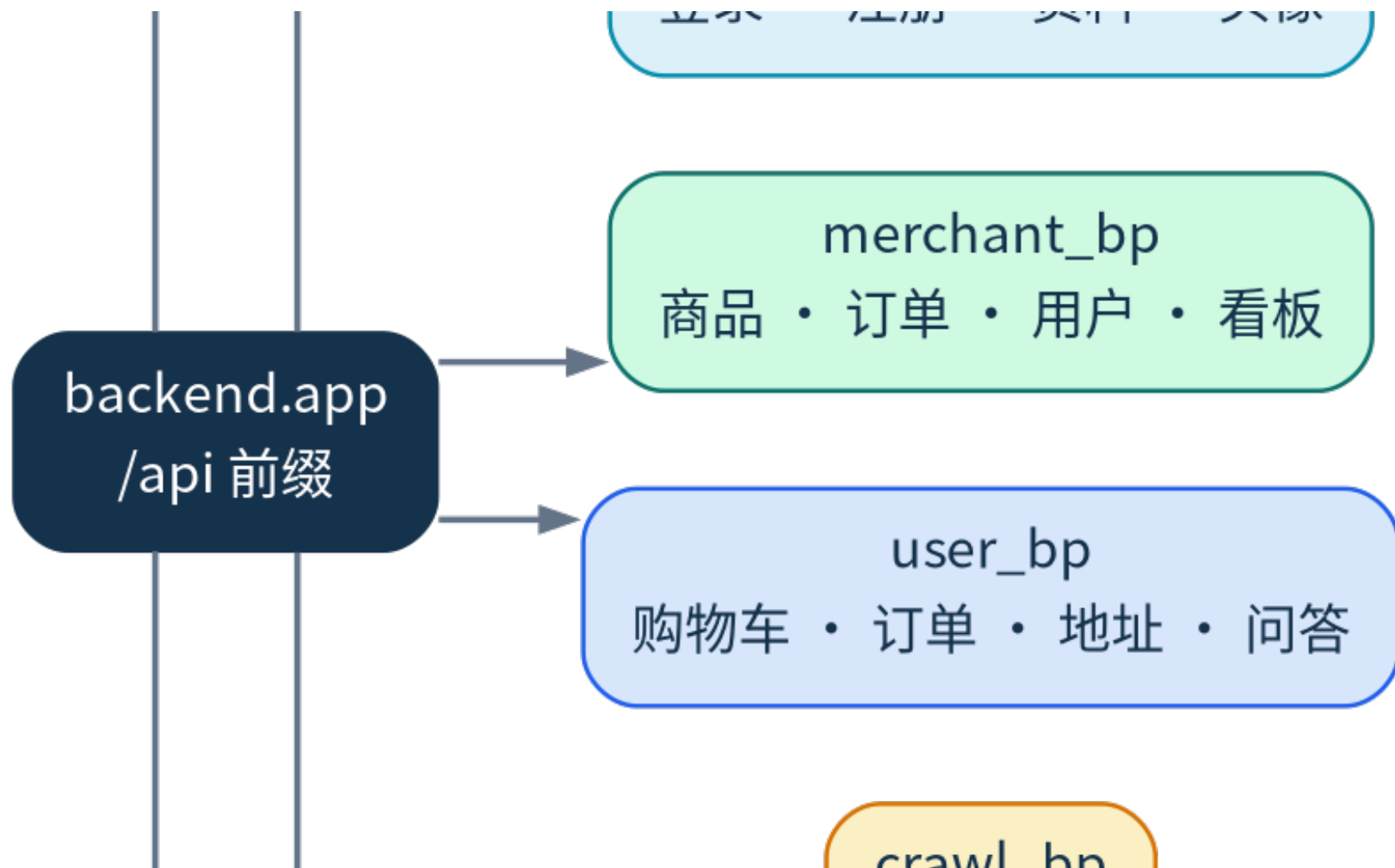
## 账号初始化

不存在时创建用户与商家账号。

## 模块装配

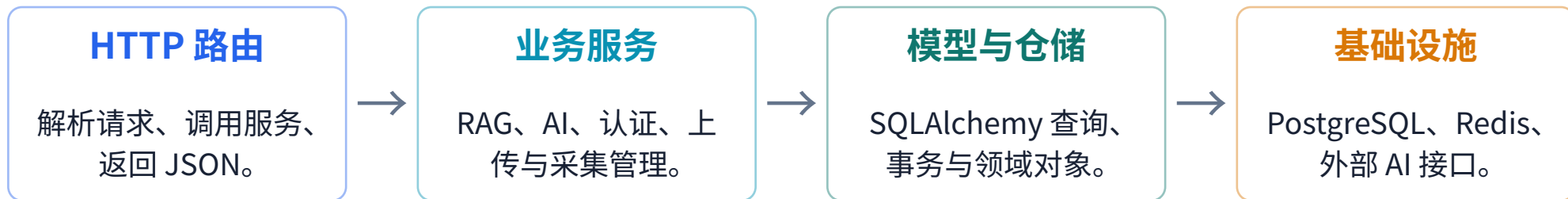
BLUEPRINTS 集中注册全部 API 域。

# API 蓝图组织



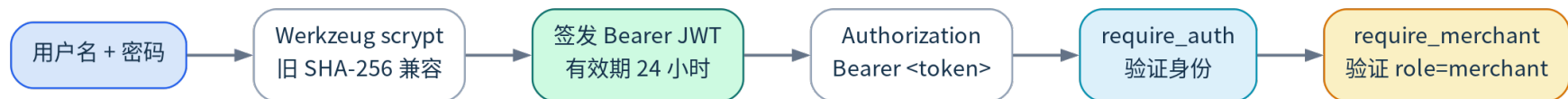
六个蓝图共享 /api 前缀；OpenAPI 页面由独立 docs\_bp 提供。路由按业务域拆分，应用工厂只负责装配。

# 分层调用关系



依赖方向由接口层指向服务层和数据层，业务模块之间通过清晰职责协作

# JWT 认证链路



## 身份校验

`require_auth` 验证 Bearer JWT 并读取用户声明。

## 角色校验

`require_merchant` 在身份通过后检查 `role=merchant`。

# 密码、密钥与跨域

## 密码存储

新密码使用 Werkzeug script 哈希；数据库不保存明文。

## 兼容验证

认证服务可读取旧 SHA-256 格式，但不会继续写入旧格式。

## JWT 密钥

生产环境使用不少于 32 字符的随机 JWT\_SECRET。

## Token 规则

仅从 Authorization Header 读取；访问令牌有效期 24 小时。

## CORS 规则

仅覆盖 /api/\*；允许来源由 CORS\_ORIGINS 集中配置。

# 数据库连接选择



空 `DATABASE_URL` 使用 instance SQLite；PostgreSQL 地址统一使用 psycopg 3。容器环境固定连接 `postgres:5432`。



# 数据库迁移策略

## 开发数据库

`init_db()` 创建本地 SQLite 表，支持首次启动和轻量开发。

```
python -c "from backend.database import init_db; init_db()"
```

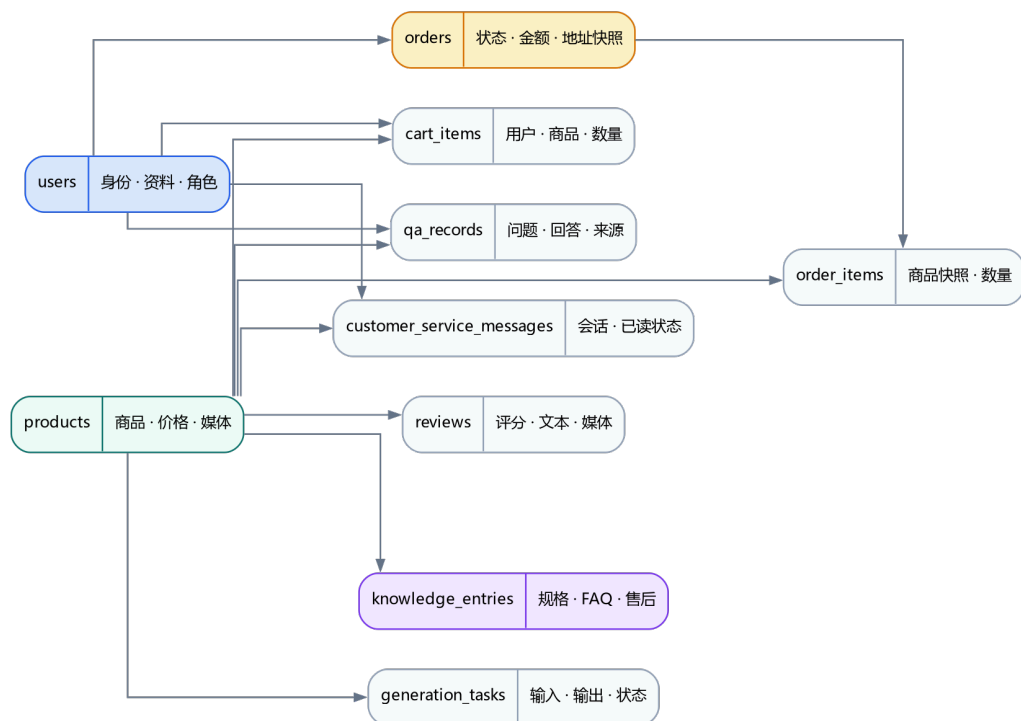
## 结构迁移

数据库结构变化由 Alembic migration 表达，容器启动先执行 `upgrade head`。

```
alembic upgrade head
```

基线迁移 `0001_schema_baseline.py` 同时兼容空数据库和已有旧表。

# 数据模型覆盖业务闭环



**用户域** 身份、地址、收藏、历史

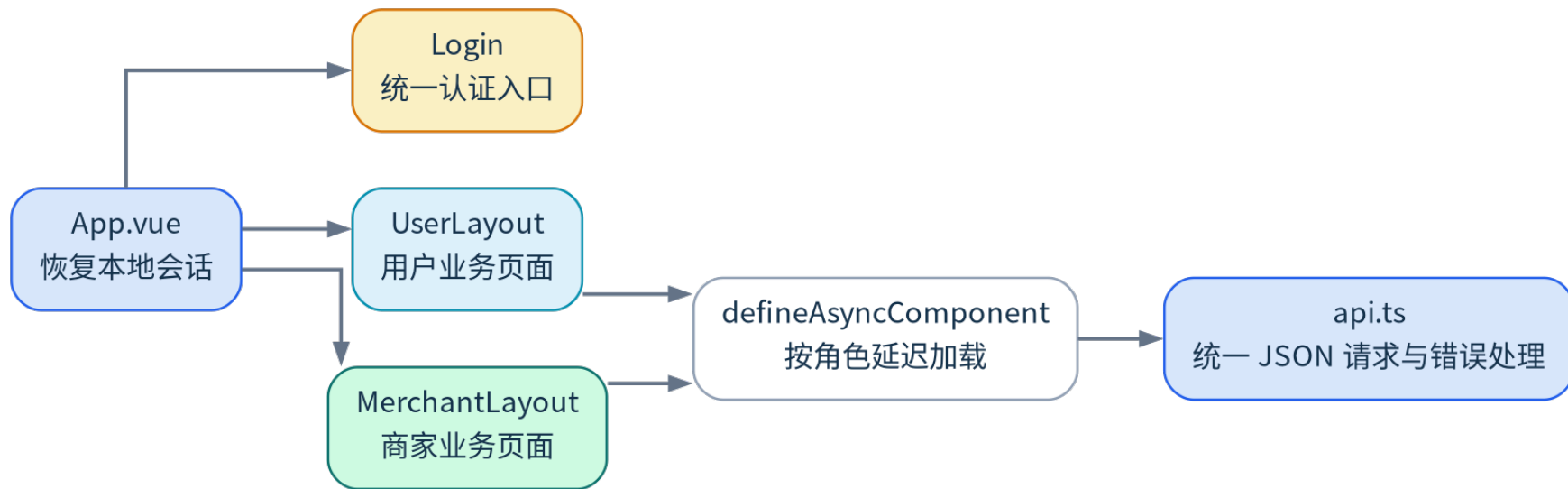
**商品域** 商品、评价、媒体

**交易域** 购物车、订单、订单项

**知识域** 知识条目、问答记录

**服务域** 生成任务、客服消息

# 前端运行结构



App.vue 恢复本地会话并按角色加载布局；业务页面通过 `defineAsyncComponent` 延迟加载。

# 前端技术职责

## Ant Design Vue

表单、菜单、卡片、上传、弹窗和消息反馈。

## Tailwind CSS

页面布局、间距、断点与响应式规则。

## Motion for Vue

登录、角色切换、页面进入退出与悬浮反馈。

## theme.ts

颜色、圆角、控件高度与动画参数的统一入口。

# 同源 API 请求路径



前端镜像不写死后端地址；只有前后端跨域部署时使用 `VITE_API_BASE_URL`。

# 按需加载与导出

## 业务页面

`defineAsyncComponent` 按角色和页面加载代码。

## 图片导出

`html2canvas` 仅在导出看板图片时加载。

## 表格导出

`write-excel-file` 仅在导出 XLSX 时加载。

首屏只包含登录与基础框架，后台页面和导出依赖不进入初始执行路径

# AI Provider 选择

## Mock

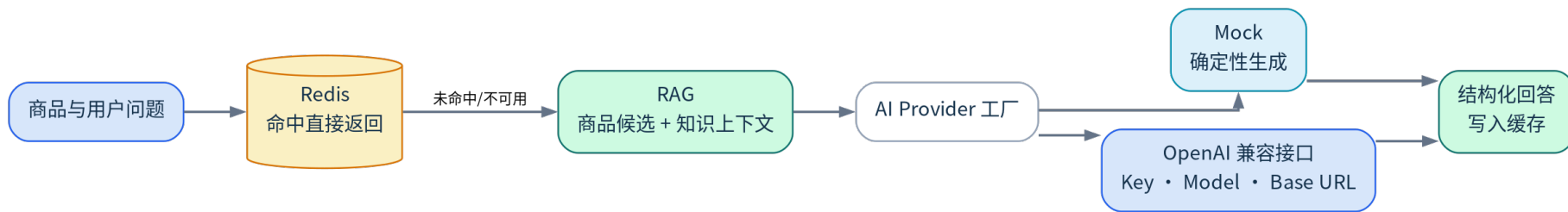
不依赖外网；输入相同则结果可重复；覆盖文案、推荐、问答、评论分析和直播脚本。

## OpenAI-compatible

读取 API Key、模型名称和可选 Base URL；业务层通过统一接口调用。

AI\_PROVIDER=mock | openai

# RAG 与缓存协作



RAG 从商品数据构建候选上下文；Redis 减少重复生成；Redis 连接失败时请求仍走完整业务链路。



# AI 请求生命周期



## 输入

Pydantic 校验商品、需求和评论参数。

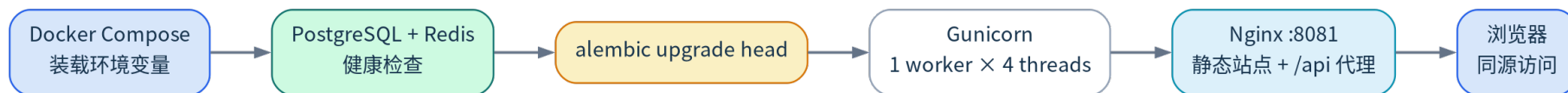
## 生成

商品与知识上下文交给 Provider。

## 结果

结构化输出写入任务与问答记录。

# 容器启动链路



Compose 等待 PostgreSQL 与 Redis 健康；后端执行迁移后启动 Gunicorn；Nginx 统一提供站点与 API。

# 容器职责

## postgres

PostgreSQL 16；数据写入 postgres\_data volume。

## redis

Redis 7；为重复生成和热点数据提供缓存。

## backend

Alembic + Gunicorn；宿主机绑定 127.0.0.1:8000。

## frontend

Nginx 静态站点与反向代理；默认端口 8081。

# 环境变量分组

| 分组  | 变量                                 | 作用                     |
|-----|------------------------------------|------------------------|
| 应用  | APP_ENV • APP_PORT • WEB_PORT      | 运行模式与服务端口              |
| 数据库 | DATABASE_URL                       | SQLite 或 PostgreSQL 连接 |
| 缓存  | REDIS_URL                          | Redis 连接地址             |
| 安全  | JWT_SECRET • CORS_ORIGINS          | Token 签名与允许来源          |
| 并发  | WEB_CONCURRENCY • GUNICORN_THREADS | Gunicorn 进程与线程         |
| AI  | AI_PROVIDER • AI_API_KEY           | 提供者选择与访问凭据             |
| 模型  | AI_MODEL • AI_BASE_URL             | 模型名称与兼容接口地址            |

# 端口与并发模型

**8081**

Nginx 对外入口

**8000**

Flask / Gunicorn

**5432**

PostgreSQL 容器

**Gunicorn**

默认 1 个 worker，每个 worker 4 个线程。

**网络边界**

数据库与后端端口绑定 127.0.0.1；浏览器只访问 Nginx。

# 健康检查与运行入口

## 服务检查

```
curl http://localhost:8000/health
```

```
python -m flask --app backend.app routes
```

## 容器检查

```
docker compose ps
```

```
docker compose logs -f backend
```

/health 返回服务版本、运行环境、数据库配置、Redis 配置和 AI Provider 摘要。

# 运行与维护命令

## 迁移

```
alembic upgrade head
```

```
alembic current
```

```
alembic revision -autogenerate
```

## 构建与配置

```
npm run build
```

```
docker compose config -quiet
```

```
ruff check backend tests migrations scripts
```

# 架构特性

## 清晰边界

Nginx、API、数据、缓存和 AI 各自承担单一职责。

## 环境一致

开发与容器共享相同 API、模型和配置结构。

## 安全隔离

JWT、角色校验、CORS 与本机端口绑定共同控制访问。

## 数据可迁移

SQLAlchemy 负责映射，Alembic 负责结构版本。

## 智能可替换

Mock 与 OpenAI-compatible 共用 Provider 契约。

## 缓存可降级

Redis 不可用时业务请求仍能完成。



# 总结

## 前端

Vue 单页应用按角色加载，通过同源 /api 访问服务。

## 后端

Flask 应用工厂装配蓝图、认证、数据与业务服务。

## 基础设施

PostgreSQL、Redis、Nginx 与 Gunicorn 由 Compose 协作。

**分层清晰 · 配置集中 · 数据可迁移 · 服务可替换**

Q & A